



Section 3 / 24

CPU Architecture

The Central Processing Unit (CPU) is the main processing unit within a computer. The CPU contains both the **Control Unit** (CU), which is in charge of moving and controlling data, and the **Arithmetic/Logic Unit** (ALU), which is in charge of performing various arithmetics and logical calculations as requested by a program through the assembly instructions.

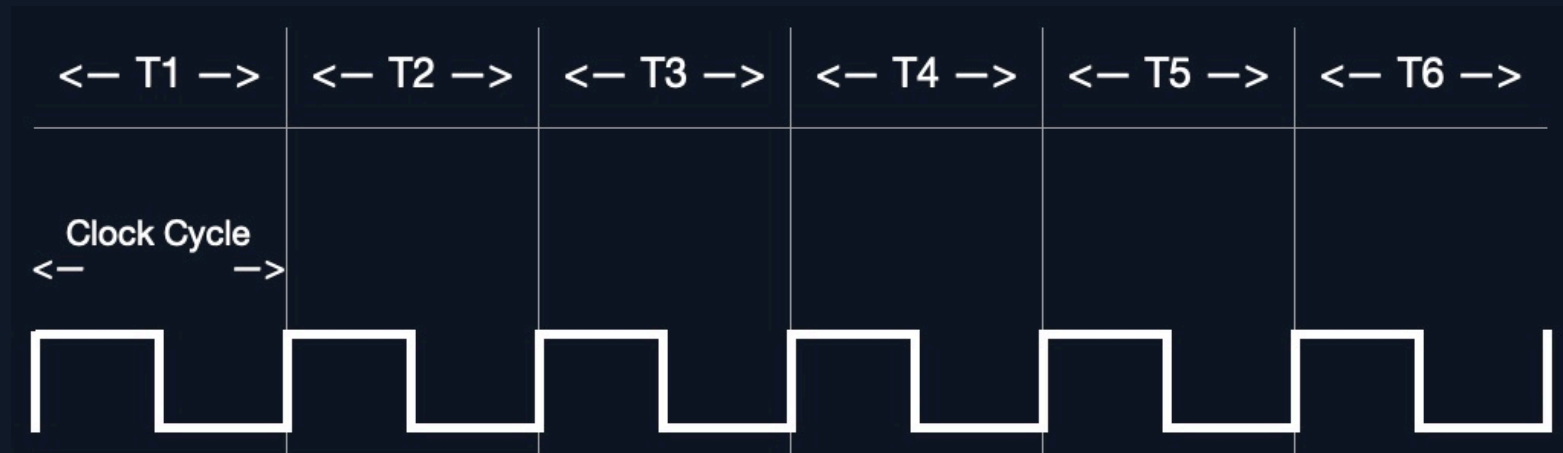
The manner in which and how efficiently a CPU processes its instructions depends on its **Instruction Set Architecture** (ISA). There are multiple ISA's in the industry, each having its way of processing data. **RISC** architecture is based on processing **more simple instructions**, which takes **more cycles**, but **each cycle is shorter and takes less power**. The **CISC** architecture is based on **fewer, more complex instructions**, which can finish the requested instructions in **fewer cycles**, but **each instruction takes more time and power to be processed**.

Let us take a look at both **RISC** and **CISC**, and learn more about instructions cycles and registers.

Clock Speed & Clock Cycle

Each CPU has a clock speed that indicates its overall speed. Every tick of the clock runs a clock cycle that processes a basic instruction, such as fetching an address or storing an address. Specifically, this is done by the CU or ALU.

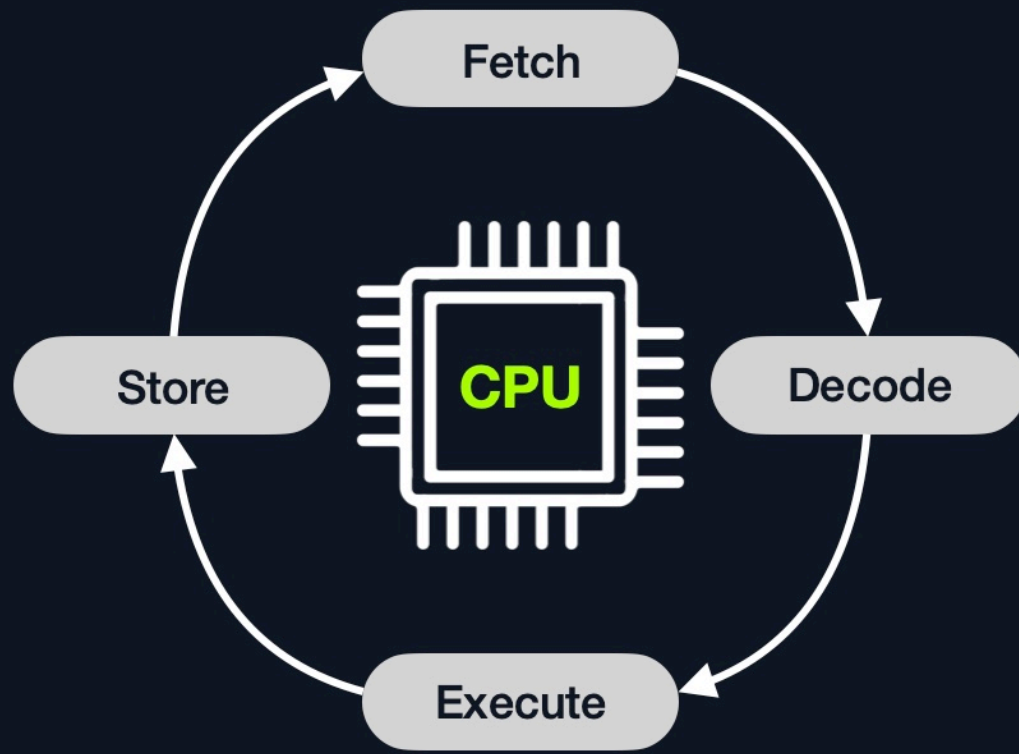
The frequency in which the cycles occur is counted in cycles per second (**Hertz**). If a CPU has a speed of 3.0 GHz, it can run 3 billion cycles every second (per core).



Modern processors have a multi-core design, allowing them to have multiple cycles at the same time.

Instruction Cycle

An **Instruction Cycle** is the cycle it takes the CPU to process a single machine instruction.



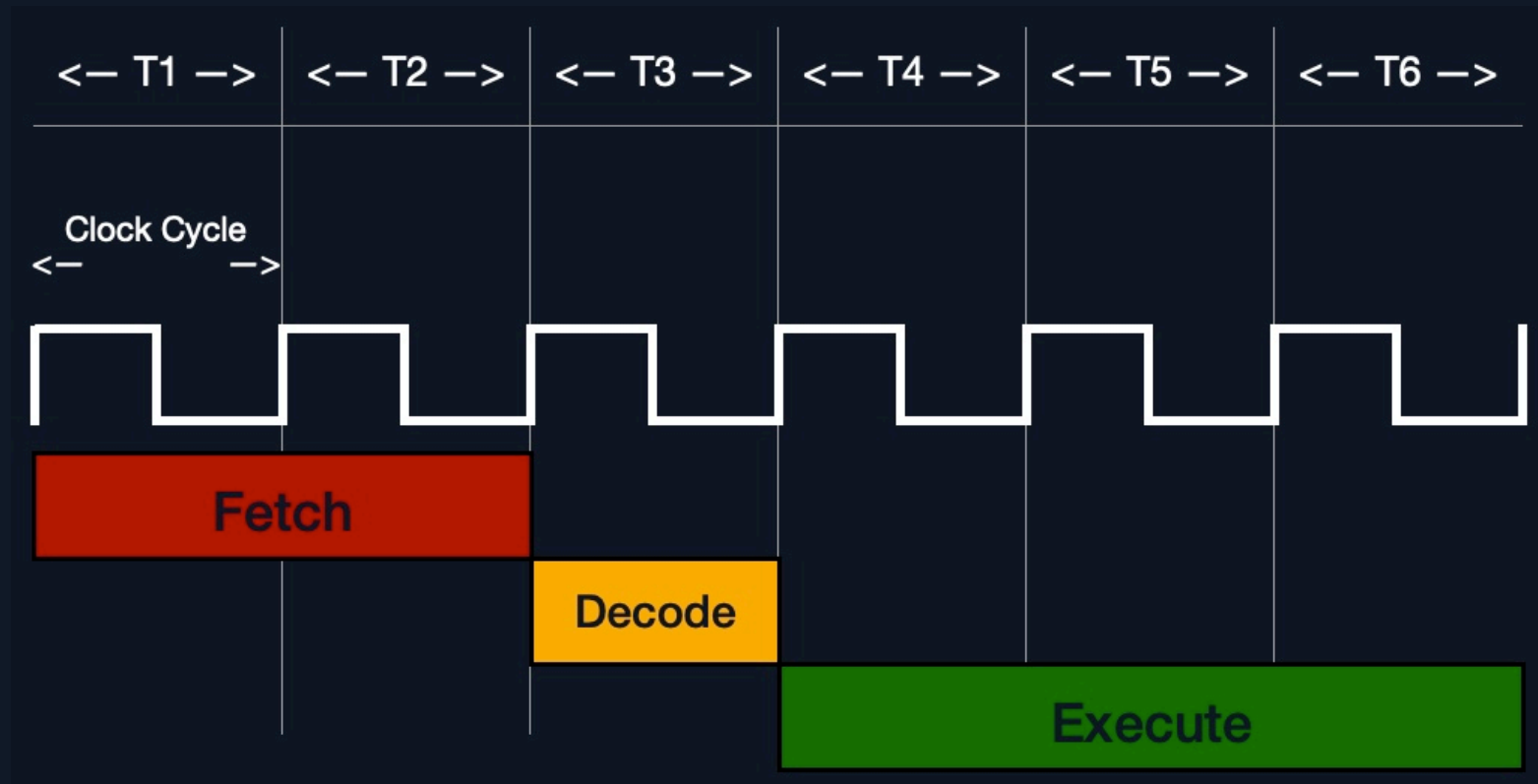
An instruction cycle consists of four stages: `Fetch` , `Decode` , `Execute` , and `Store` :

Instruction	Description
1. <code>Fetch</code>	Takes the next instruction's address from the <code>Instruction Address Register</code> (IAR), which tells it where the next instruction is located.
2. <code>Decode</code>	Takes the instruction from the IAR, and decodes it from binary to see what is required to be executed.
3. <code>Execute</code>	Fetch instruction operands from register/memory, and process the instruction in the <code>ALU</code> or <code>CU</code> .

Instruction	Description
4. Store	Store the new value in the destination operand.

All of the stages in the instruction cycle are carried out by the Control Unit, except when arithmetic instructions need to be executed "add, sub, ..etc", which are executed by the ALU.

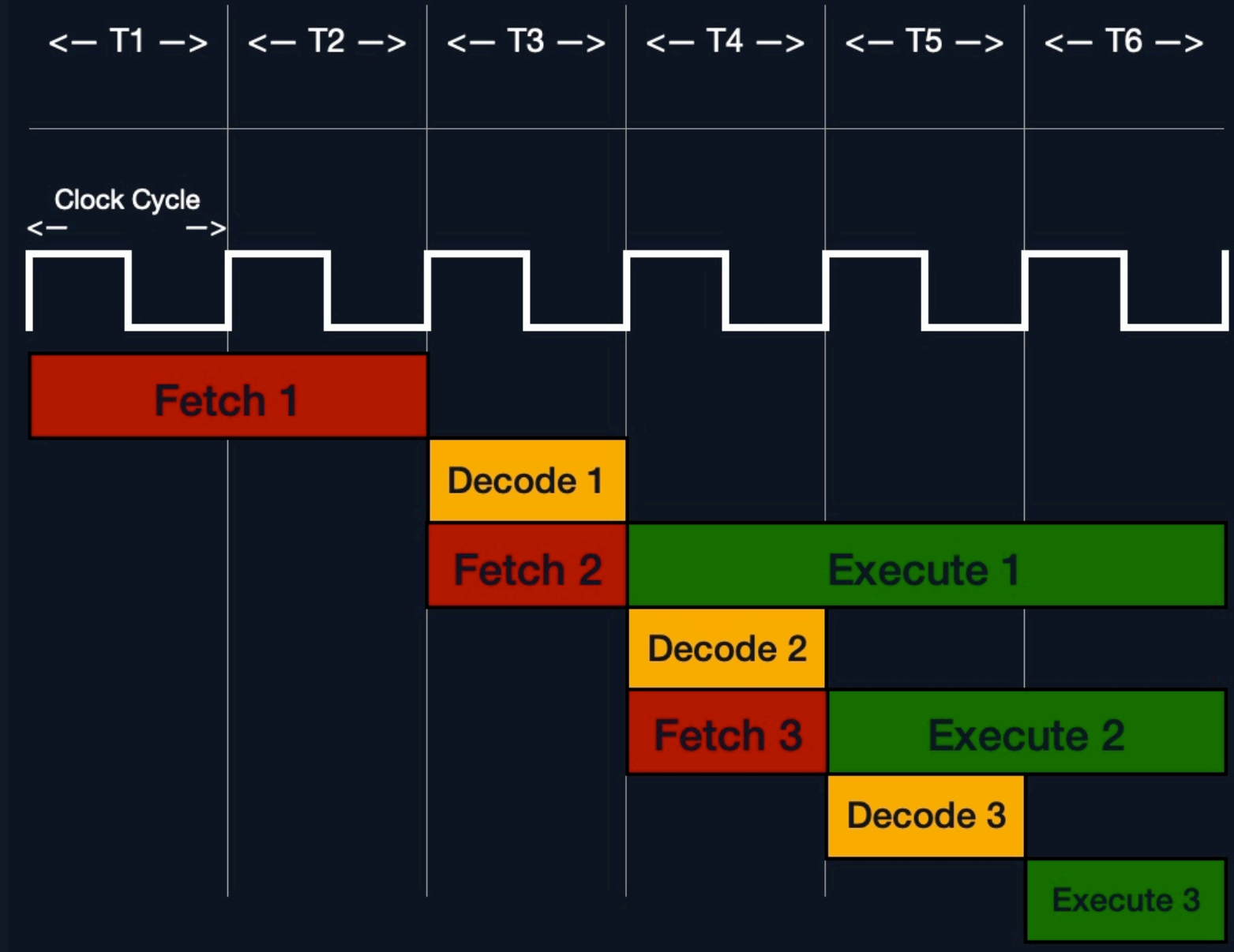
Each Instruction Cycle takes multiple clock cycles to finish, depending on the CPU architecture and the complexity of the instruction. Once a single instruction cycle ends, the CU increments to the next instruction and runs the same cycle on it, and so on.



For example, if we were to execute the assembly instruction `add rax, 1`, it would run through an instruction cycle:

1. Fetch the instruction from the `rip` register, `48 83 C0 01` (in binary).
2. Decode '`48 83 C0 01`' to know it needs to perform an `add` of `1` to the value at `rax`.
3. Get the current value at `rax` (by `CU`), add `1` to it (by the `ALU`).
4. Store the new value back to `rax`.

In the past, processors used to process instructions sequentially, so they had to wait for one instruction to finish to start the next. On the other hand, modern processors can process multiple instructions in parallel by having multiple instruction/clock cycles running at the same time. This is made possible by having a multi-thread and multi-core design.



Processor Specific

As previously mentioned, each processor understands a different set of instructions. For example, while an Intel processor based on the 64-bit x86 architecture may interpret the machine code `4883C001` as `add rax, 1`, an ARM processor translates the same machine code as the `biceq`

`r8, r0, r8, asr #6` instruction. As we can see the same machine code performs an entirely different instruction on each processor.

This is because each processor type has a different low-level assembly language architecture known as **Instruction Set Architectures** (ISA). For example, the `add` instruction seen above, `add rax, 1` is for Intel x86 64-bit processors. The same instruction written for the ARM processor assembly language is represented as `add r1, r1, 1`.

It is important to understand that each processor has its own set of instructions and corresponding machine code.

Furthermore, a single Instruction Set Architecture may have several syntax interpretations for the same assembly code. For example, the above `add` instruction is based on the x86 architecture, which is supported by multiple processors like Intel, AMD, and legacy AT&T processors. The instruction is written as `add rax, 1` with Intel syntax, and written as `addb $0x1,%rax` with AT&T syntax.

As we can see, even though we can tell that both instructions are similar and do the same thing, their syntax is different, and the locations of the source and destination operands are swapped as well. Still, both codes assemble the same machine code and perform the same instruction.

So, each processor type has its Instruction Set Architectures, and each architecture can be further represented in several syntax formats

This module will focus mainly on the Intel x86 64-bit assembly language (also known as x86_64 and AMD64), as the majority of modern computers and servers run on this processor architecture. We will be using the Intel syntax as well.

If we want to know whether our Linux system supports `x86_64` architecture, we can use the

`lscpu` command:

```
kimvutrinh@htb[/htb]$ lscpu
```

```
Architecture:            x86_64
CPU op-mode(s):          32-bit, 64-bit
Byte Order:              Little Endian
```

```
<SNIP>
```

As we can see in the above output, the CPU architecture is `x86_64`, and supports 32-bit and 64-bit. The byte order is Little Endian. We can also use the `uname -m` command to get the CPU architecture. We will discuss the two most common Instruction Set Architectures in the next section: `CISC` and `RISC`.

[< Previous](#)

Section 3 / 24

 +20

[Next](#)[Mark Complete & Next](#)